

ABC-FHE: A Resource-Efficient Accelerator Enabling Bootstrappable Parameters for Client-Side Fully Homomorphic Encryption

Sungwoong Yune Hyejeong Lee Adiwena Putra Hyunjun Cho
Cuong Duong Manh Jaeho Jeon Joo-Young Kim

School of Electrical Engineering, KAIST

{imwooong, hjlee8877, adiwena.research, h.cho, cuongdm1410, math15738, jooyoung1203}@kaist.ac.kr

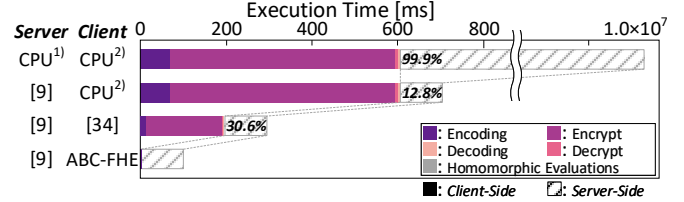
Abstract—As the demand for privacy-preserving computation continues to grow, fully homomorphic encryption (FHE)—which enables continuous computation on encrypted data—has become a critical solution. However, its adoption is hindered by significant computational overhead, requiring 10000-fold more computation compared to plaintext processing. Recent advancements in FHE accelerators have successfully improved server-side performance, but client-side computations remain a bottleneck, particularly under bootstrappable parameter configurations, which involve combinations of encoding, encrypt, decoding, and decrypt for large-sized parameters. To address this challenge, we propose ABC-FHE, an area- and power-efficient FHE accelerator that supports bootstrappable parameters on the client side. ABC-FHE employs a streaming architecture to maximize performance density, minimize area usage, and reduce off-chip memory access. Key innovations include a reconfigurable Fourier engine capable of switching between NTT and FFT modes. Additionally, an on-chip pseudo-random number generator and a unified on-the-fly twiddle factor generator significantly reduce memory demands, while optimized task scheduling enhances the CKKS client-side processing, achieving reduced latency. Overall, ABC-FHE occupies a die area of 28.638 mm² and consumes 5.654 W of power in 28 nm technology. It delivers significant performance improvements, achieving a 1112 $\times$ speed-up in encoding and encryption execution time compared to a CPU, and 214 $\times$ over the state-of-the-art client-side accelerator. For decoding and decryption, it achieves a 963 $\times$ speed-up over the CPU and 82 $\times$ over the state-of-the-art accelerator.

Index Terms—accelerator, bootstrappable parameter, client-side, fully homomorphic encryption

I. INTRODUCTION

In recent years, personal data from edge devices, such as PCs and mobile phones, has been increasingly sent to centralized servers for advanced computation with the paradigm of cloud computing [1]. This trend is evident in real-time AI applications that have excessive computational requirements, including chatbot [2], voice assistant [3], and image recognition [4]. However, this raises privacy concerns, as users should share their personal data on cloud servers. Fully homomorphic encryption (FHE) is considered the solution by allowing computations on data while it remains encrypted, thereby preserving privacy.

While FHE provides a solution for privacy-preserving computations, it also introduces substantial computational challenges [5]. Specifically, in the CKKS [6] scheme, a popular choice for AI-based applications, the ciphertext is a high-degree polynomial (ranging from degree 2^{14} to 2^{16}) with each coefficient represented as a high-precision integer of hundreds to thousands of bits. These coefficients are decomposed in the residue number system (RNS), reducing them to approxi-



1) : Measured with dual Intel Xeon Platinum 8280 @3.2GHz 112 cores

2) : Measured with Intel Core i7-12700 @2.1GHz 1 core

Fig. 1. Execution time breakdown of overall client-side and server-side operations.

mately 50–60 bits and enabling more efficient arithmetic operations. To accelerate these computations, the number theoretic transform (NTT) is often used, reducing the complexity of polynomial multiplication from N^2 to $N \log(N)$. Despite this algorithmic enhancement, workloads within the FHE domain remain up to 10,000 $\times$ slower than their plaintext counterparts due to the inherent computational complexity, emphasizing the need for dedicated hardware accelerators to achieve the demanding performance of modern applications [7].

Among hardware accelerators, the majority have focused on achieving advancements on the server-side [8]–[12]. However, the need for client-side accelerators has become increasingly evident for two main reasons. First, server-side computation is no longer the primary bottleneck due to drastically reduced computation times by server-side FHE accelerators. For example, as shown in Figure 1, executing FHE operations on the ResNet20 model reveals that encryption and decryption tasks on the client-side account for 69.4% of the total execution time using a state-of-the-art (SOTA) client-side accelerator [13], whereas computations on the server-side account for only 30.6% with a SOTA server-side ASIC accelerator [12]. Second, client-side computations involve distinct workload characteristics, requiring both fast Fourier transform (FFT) and NTT for processing. These computations depend on floating-point and integer operations, respectively, and are typically performed in a non-repetitive manner, presenting unique challenges compared to server-side workloads.

Several prior works have explored client-side FHE accelerators [13]–[16], but they suffer from several limitations. First, these accelerators are constrained to small FHE parameters (e.g., $N = 2^{13}$), rendering them incapable of supporting bootstrapping operations that require larger polynomial degrees, typically at least $N = 2^{14}$ and often extending to $N = 2^{16}$. Second, prior works using non-streaming architectures encounter severe DRAM bandwidth limitations. Even if throughput is increased to accelerate computations, the

resulting larger data output per cycle exceeds the available DRAM bandwidth, leading to latency overhead rather than performance gains. Third, the SOTA work [13] intensifies the DRAM bandwidth bottleneck by directly fetching parameters from DRAM, further increasing memory bandwidth demands and exacerbating latency overhead.

To address these issues, we propose ABC-FHE, a resource-efficient FHE accelerator specifically designed to support bootstrappable parameters for client-side applications. First, it is tailored to accommodate bootstrappable parameters, aligning with the practical requirements of FHE deployment. Second, it adopts a streaming architecture, addressing memory bottlenecks associated with large bootstrappable parameters with efficient memory utilization and seamless operation. Third, it incorporates an on-chip pseudo-random number generator (PRNG) and an on-the-fly twiddle factor generator (OTF TF Gen) to generate random values and twiddle factors directly on-chip, reducing reliance on external memory. Furthermore, it introduces methods to minimize the area overhead caused by the structural characteristics of the streaming architecture.

To summarize, we make the following contributions:

- We analyze the workload of the client-side CKKS scheme, identifying distinct workload characteristics and imbalances between encryption-related tasks (IFFT + NTT) and decryption-related tasks (FFT + INTT).
- To meet latency requirements while accommodating diverse workload characteristics, we propose a streaming architecture with a reconfigurable compute engine that supports both floating-point complex-number-based I/FFT and integer-based I/NTT operations.
- To further optimize area efficiency, we introduce the unified on-the-fly twiddle factor generator (OTF TF Gen), enabling the NTT/FFT engine to share a single hardware unit for twiddle factor generation. Additionally, we provide a methodology for selecting optimal prime numbers to facilitate hardware-friendly modular multiplication.
- Overall, ABC-FHE achieves a speed-up of up to 1112 $\times$ for encoding and encryption and up to 963 $\times$ for decoding and decryption compared to a CPU, while delivering up to 214 $\times$ and 82 $\times$ improvements, respectively, over SOTA accelerators.

II. BACKGROUND

For the remainder of this paper, we use the following notations: In representation of polynomial, n denotes the degree, N represents the polynomial degree, Q refers to the prime number, and P in the FFT architecture indicates the number of parallelism lanes.

A. Fully Homomorphic Encryption

FHE achieves security by injecting error into the computation process, requiring a specialized operation called bootstrapping to reset the accumulated error during consecutive computations. In the CKKS scheme, this error is managed as “levels” which decrease as error accumulates through operations. Bootstrapping restores the levels and reduces the

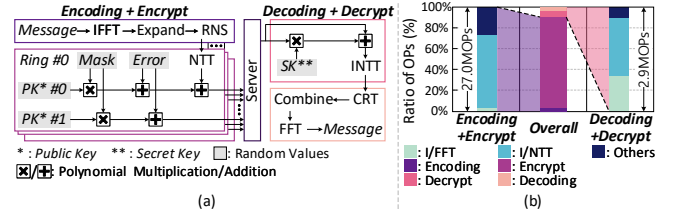


Fig. 2. Workload analysis of CKKS client-side operations: (a) Operational flow and (b) Ratio of operations for 12 level encoding/encryption and 1 level decoding/decryption supporting polynomial degree 2^{16} .

error; however, the process also consumes levels as part of its execution. To minimize the frequency of this computationally intensive bootstrapping, higher levels are necessary. Achieving a high level while ensuring the 128-bit security standard, which is considered secure in modern cryptographic systems, requires large polynomial degrees ranging from 2^{14} to 2^{16} [17].

B. Fourier-like Transform

In FHE, polynomial multiplication is required, which has a complexity of N^2 . By transforming from the time domain to the frequency domain using fast transforms, this complexity can be reduced to $N \log(N)$. Two types of transformations are commonly used in FHE: NTT and FFT, which perform complex-number and modular computations, respectively. Both transforms share the same fundamental equation but differ in their respective weights, known as the twiddle factors. Specifically, NTT operates within a prime modular ring (Q), while FFT operates on the unit circle in the complex plane. The equation for Fourier-like transform is as follows :

$$A[k] = \sum_{i=0}^{N-1} a(i) \times W_N^{ki} \quad \dots (1)$$

where W_N represents the twiddle factor for each transformation.

C. Hardware for Fourier-like Transform

Even with complexity optimization through FFT, N can reach up to 2^{16} in CKKS, which still requires high computational capacity. Many prior works have sought to support the FFT algorithm through dedicated hardware [18]–[20]. There are two main approaches to optimizing FFT architecture.

Spatial Approach. To maximize throughput, increasing the number of computation lanes (P) is an obvious approach, as it accelerates processing at the cost of additional area. However, on the client-side, where careful external memory accesses (EMA) handling is required, generating a large amount of output data per cycle can lead to additional stalls. Therefore, the number of lanes must be chosen carefully, as a larger number is not always optimal.

Temporal Approach. The limitation on the number of lanes necessitates an architecture with multiple stages. Each stage operates in a pipelined manner, allowing the Fourier-like transform to achieve constant throughput per cycle, eliminating any latency disparities compared to non-pipelined designs. This makes a pipelined NTT architecture more suitable for client-side hardware.

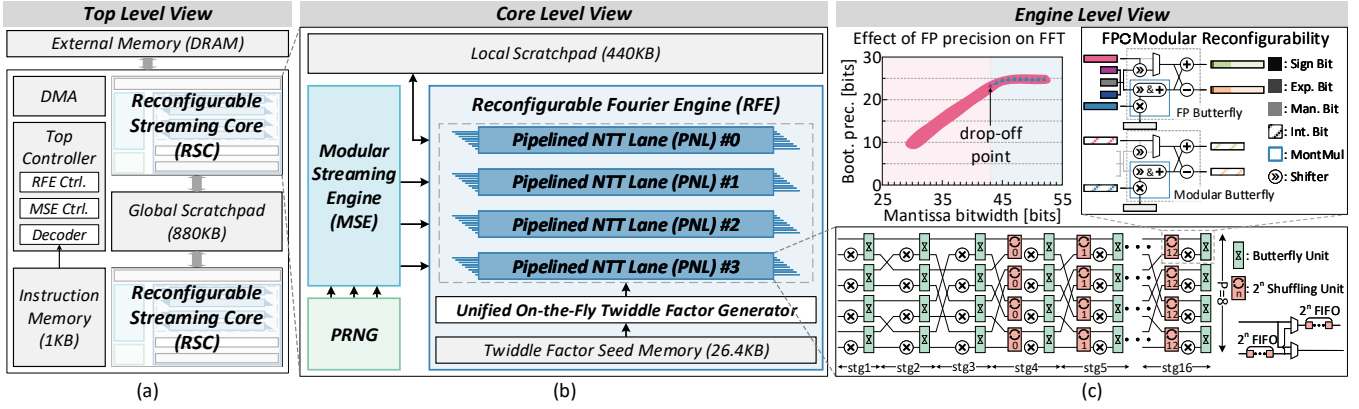


Fig. 3. Overall architecture of ABC-FHE (a) Top level view (b) Core level view (c) Engine level view.

D. Workload Characteristic of Client-Side FHE

Figure 2 highlights two key characteristics of CKKS client-side operations when computing a single ciphertext. First, as shown in Figure 2 (a), encoding and encryption require simultaneous support for IFFT and NTT to convert messages into ciphertexts, whereas decoding and decryption require FFT and INTT for reversing ciphertexts into messages. Second, as illustrated in Figure 2 (b), the number of operations for encoding and encryption is nearly ten times greater than for decoding and decryption. This imbalance underscores the inefficiency of implementing separate modules for encryption and decryption.

III. ABC-FHE ARCHITECTURE

To address the limitations of prior works that are restricted to small parameters [13]–[16], we designed an accelerator capable of efficiently handling larger parameters. In resource-constrained client-side environments, simply increasing throughput without regard for area and resource limitations can lead to memory bottlenecks or an unrealistic demand for on-chip memory. Thus, ABC-FHE employs a streaming architecture to maintain consistent throughput while optimizing area, addressing these challenges effectively.

Figure 3 (a) shows the overall architecture of ABC-FHE. At the top level, two homogeneous reconfigurable streaming cores (RSC) enable the streaming-based design to stably handle either message encoding or ciphertext decoding. The reconfigurable nature of RSC allows for three operational modes: doubling the throughput for encrypt, doubling the throughput for decrypt, or simultaneously performing encrypt and decrypt. Additionally, a global scratchpad is included to fetch messages or ciphertext from external memory, ensuring continuous data flow to the two cores.

Figure 3 (b) presents the core-level architecture of RSC, optimized for efficient streaming operations across its three operational modes. At its core, RSC incorporates two primary engines to enable seamless functionality: the reconfigurable Fourier engine (RFE) and the modular streaming engine (MSE). The RFE, equipped with four pipelined NTT lanes (PNLs), supports both I/NTT and I/FFT computations to maximize throughput for Fourier-like transforms. Complementing this, the MSE handles additional SIMD-like oper-

ations, including RNS, Chinese Remainder Theorem (CRT) computations for inverse RNS, and element-wise additions and multiplications. To minimize EMA, the architecture emphasizes on-chip value generation. Specifically, an on-chip PRNG produces random values such as masks, errors, and keys for ciphertext generation, eliminating the need to fetch them from external memory. While the OTF TF Gen, shared among the four PNLs, dynamically generates twiddle factors for Fourier-like transforms.

Figure 3 (c) highlights the RFE, the central component of RSC, designed to efficiently support both I/FFT and I/NTT operations through optimized bitwidth configurations. For NTT, recent studies addressed the challenge of substantial bitwidth requirements in CKKS by dividing data into smaller 32–36 bit segments across multiple levels while doubling the number of levels, a solution proven feasible for CKKS [21]. This allows ABC-FHE to adopt a smaller datapath bitwidth, optimizing efficiency. For I/FFT, required precision is measured by bootstrapping precision (Boot. prec.), which indicates the bit precision needed after server-side bootstrapping to maintain model accuracy. Prior research [9] established that a Boot. prec. above 19.29 bits sufficiently supports AI models without significant accuracy degradation. To minimize the FFT datapath, which traditionally relies on the 64-bit floating-point (FP64) data type, we iteratively reduced the floating-point mantissa bitwidth and evaluated Boot. prec. at each step. As shown in the graph in Figure 3 (c), maintaining at least 43 mantissa bits results in a Boot. prec. of 23.39 bits, exceeding the required threshold. Accordingly, we implemented a custom 55-bit floating-point (FP55) format for I/FFT and a 44-bit modular operation for I/NTT, designed in a reconfigurable manner to maximize efficiency for both computation types.

IV. MICROARCHITECTURE

A. Reconfigurable Fourier Engine (RFE)

ABC-FHE employs the NTT algorithm to perform polynomial multiplication. To achieve acceleration through increased throughput while maintaining a consistent rate, a pipelined NTT design with a multi-path delay commutator (MDC)-based backbone [18] ($P=8$) was chosen. However, the increased number of butterfly units required for this design results in larger overall area. To address this, ABC-FHE

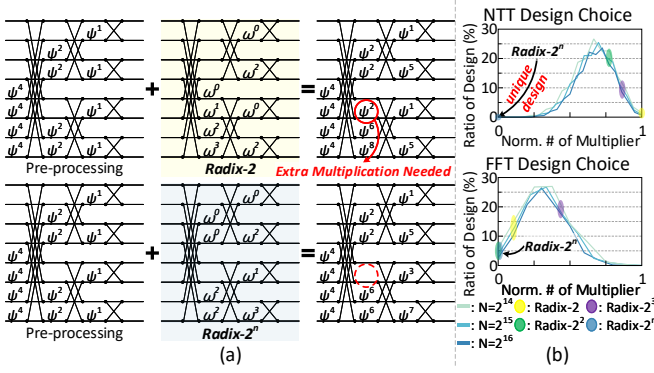


Fig. 4. (a) Twiddle factor scheduling in NTT signal flow graph (b) Distribution of multiplier counts across possible NTT/FFT design configurations.

introduces three optimizations: (1) twiddle factor scheduling to reduce the total number of multipliers, (2) an algorithm-hardware co-designed Montgomery multiplier to minimize multiplier area, and (3) leveraging reconfigurability among PNLs to support both NTT and FFT operations.

Twiddle Factor Scheduling. Research on pipelined architectures for the FFT has been extensive [22]–[24], with a particular focus on minimizing the area of computational units. In FFT, efforts to optimize area include treating the multiplication between twiddle factors and inputs through rotators, enabling the identification of optimal rotator conditions in pipelined architectures. However, in the NTT, all multipliers are unified as modular multipliers, unlike the FFT approach. This makes it essential to reduce the number of computational units. Additionally, to simplify polynomial operations in NTT, the nega-cyclic property is employed, which requires extra pre-processing and post-processing steps. The equations for pre-processing and post-processing are shown below.

$$A[k] = \sum_{n=0}^{N-1} a(n) \times W_N^{kn} \times \psi_{2N}^n \pmod{Q} \dots (2)$$

$$a[n] = N^{-1} \sum_{k=0}^{N-1} A[k] \times W_N^{-kn} \times \psi_{2N}^{-k} \pmod{Q} \dots (3)$$

$$\text{where } \psi_{2N}^2 \equiv W_N \pmod{Q}$$

The NTT algorithm can be optimized by merging pre-processing and post-processing operations with twiddle factors, as demonstrated in [25] and [26]. This merging technique eliminates the need for additional multipliers by applying a consistent pattern of twiddle factor operations across stages in the signal flow graph. By aligning the arrangement of twiddle factors with the pre-/post-processing pattern, the merging technique can be seamlessly implemented without extra computational units. Consequently, the minimum number of multipliers required for a pipelined NTT architecture is theoretically $P/2 \times \log_2 N$. Figure 4 (a) shows the distribution of twiddle factors in the signal flow graph using an 8-point example. For instance, with a radix-2 design, pre-processing results in 13 twiddle factor multiplications, while a radix-2ⁿ design reduces this to 12 multiplications. As the polynomial

degree increases, the design choice significantly impacts the total number of multiplications required.

To further optimize multiplier usage, we analyzed a range of design configurations, including less conventional options, to evaluate their overheads. Figure 4 (b) compares the overheads of different design options for NTT and FFT. Our analysis revealed that only radix-2ⁿ designs maintain the consistent twiddle factor pattern required for efficient pipelined NTT implementation, making them the most suitable choice. Consequently, we adopted radix-2ⁿ designs for both NTT and FFT operations, highlighting that our radix-2ⁿ configuration achieves a 29.7% and 22.3% reduction in number of multipliers compared to widely used radix-2 and radix-2² designs in NTT, respectively.

NTT-Friendly Montgomery Multiplier. Modular multiplication, which involves both multiplication and division by a specific number, is inherently expensive. To address this, both algorithmic optimizations [27], [28] and hardware implementations have been explored [29]–[32]. Among the common approaches, Barrett multiplication approximates division with a multiplication to simplify the operation, while Montgomery multiplication first transforms the input into the Montgomery domain via pre-processing, performs modular reduction using simpler arithmetic, and then converts it back to the original domain via post-processing. Montgomery reduction efficiently reduces modular arithmetic computations in the Montgomery domain. It operates on two integers a and b ($a, b < Q$), with a modulus Q of p_{bw} -bit bitwidth and a coprime R ($R \geq Q$).

$$T = a \times b \dots (4)$$

$$m \equiv ((T \pmod{R}) \times QInv) \pmod{R} \dots (5)$$

$$t \equiv (T - m \times Q) \pmod{R} \dots (6)$$

$$t = \begin{cases} t + Q, & \text{if } t < 0 \\ t, & \text{otherwise} \end{cases} \dots (7)$$

Montgomery multiplication is typically the simplest and most efficient. However, Montgomery reduction inherently requires three multipliers, introducing significant overhead, particularly in FHE applications with large bitwidths. To address this issue, we optimized the Montgomery reduction algorithm by modifying the computation of $QInv$ to be processed using a shift-and-add approach.

Before presenting the optimized algorithm, we define the NTT-friendly prime number and Euler's theorem as follows:

$$Q = 2^{p_{bw}} + k \cdot 2^{n+1} + 1 \dots (8)$$

$$QInv \equiv Q^{2^{n-1}-1} \pmod{2^n} \dots (9)$$

Equation (8) represents the expression of an NTT-friendly prime, where k is an arbitrary value. Equation (9) uses Euler's theorem to define $QInv$. $\equiv$ in equation (10) and (11) represent modular equivalence with 2^n . Substituting the expression for Q from Equation (8) into Equation (9) yields the following:

$$QInv \equiv \sum_{i=0}^{2^{n-1}-1} \binom{2^{n-1}-1}{i} \cdot (2^{p_{bw}} + k \cdot 2^{n+1})^i \dots (10)$$

TABLE I
AREA OF MODULAR MULTIPLIER

Algorithm	Area (μm^2)	Pipeline Stages (Cycles)
Vanilla Barrett	35054	4
Vanilla Montgomery	19255	3
NTT-Friendly Montgomery	11328	3

If k satisfies the condition $k \geq 2^{bw/2-1-n}$, with $k = \pm 2^a \pm 2^b \pm 2^c$, $QInv$ can be simplified as follows:

$$QInv \equiv -2^{p-bw} - (\pm 2^a \pm 2^b \pm 2^c) \cdot 2^{n+1} + 1 \quad \dots (11)$$

By applying this to the Montgomery algorithm, all multiplications except for the initial multiplication can be converted into shift-and-add operations. This significantly reduces the hardware complexity compared to the original implementation, which required three multipliers.

Despite reducing the number of supported primes due to this optimization, the design still sufficiently supports 20-40 encryption levels. For instance, in the case of $N = 2^{16}$, the required 32-36 bit primes amount to a total of 443, which is more than adequate to handle all practical scenarios. Table I further highlights the area and pipeline stages of different modular multipliers (MM), synthesized at 600 MHz using 28 nm process. The NTT-friendly Montgomery MM achieves a 67.7% area reduction compared to the Barrett MM and a 41.2% reduction compared to the vanilla Montgomery MM.

Reconfigurability among PNLs. To support FFT computations, complex number FP operations are required. These involve calculations such as:

$$(a + bi) \cdot (c + di) = (ac - bd) + i(ad + bc) \quad \dots (12)$$

which require four FP multiplications. To efficiently handle these operations, the modular multipliers and FP multipliers are designed to operate in a reconfigurable manner. This enables the use of four modular multipliers to support complex number FP multiplication. The same approach is applied to the unified OTF TF Gen, ensuring seamless support for generating twiddle factors.

B. Efficient Memory Handling

ABC-FHE addresses the challenge of minimizing memory requirements for FHE operations on the client-side by generating essential data on-chip, thereby reducing reliance on external memory. For client-side FHE with a polynomial degree of 2^{16} , 44-bit precision, and 24 levels, calculations estimate the need for 16.5 MB of public key storage, 8.25 MB for masks and errors, and an additional 8.25 MB for twiddle factors. Such substantial on-chip memory is impractical for client-side applications, while frequent external memory fetches require high-bandwidth solutions, such as HBM, which are unsuitable for typical client-side environments.

To address these constraints, ABC-FHE incorporates a PRNG and an unified OTF TF Gen to generate data directly on-chip. The PRNG requires only a 128-bit seed on-chip to meet the 128-bit security level, eliminating the need to store extensive precomputed values. The unified OTF TF Gen dynamically generates twiddle factors for each stage of

TABLE II
THE AREA AND POWER BREAKDOWN OF ABC-FHE

Component	Area (mm^2)	Power (W)
4x PNL	10.717	1.397
Unified OTF TF Gen	0.697	0.089
Twiddle Factor Seed Memory	0.046	0.022
MSE	0.787	0.298
PRNG	0.069	0.028
Local Scratchpad	0.658	0.323
RSC	12.973	2.156
2x RSC	25.946	4.313
Global Scratchpad	2.632	1.290
Top CTRL, DMA, Etc.	0.060	0.051
Total	28.638	5.654

the pipelined NTT lane using a compact twiddle factor seed and step size, requiring approximately 27 KB, which reduces on-chip memory requirements by over 99.9%. This efficient approach not only significantly reduces memory overhead but also enables scalable and practical FHE acceleration for client-side environments.

V. EVALUTATION

A. Hardware Modeling

We synthesize ABC-FHE's functional units, scratchpads, and interconnections using a 28nm process design kit, with a focus on minimizing area and power consumption while maintaining a 600 MHz clock frequency. All functional unit datapaths, including the RFE and MSE, support both 44-bit integer and 55-bit floating-point operations. The global scratchpad, designed as a double-buffered, single-port, multi-bank 256-bit width SRAM with a total capacity of 880 KB, stores messages received from the host and processed ciphertexts sent to the server. Assuming LPDDR5, which is commonly used in client-side environments with a bandwidth of 68.4 GB/s, this architecture effectively manages message fetching and ciphertext transmission. Additionally, each RSC incorporates a local scratchpad, implemented as a single-port, multi-bank 256-bit width SRAM. For PNL implementation, the shuffling unit requires FIFOs whose capacity doubles with each stage; these are also implemented as double-buffered SRAMs to minimize area overhead. Table II summarizes the synthesis results of ABC-FHE with two RSC instances. The overall design occupies 28.638 mm^2 and consumes up to 5.654 W of power. Based on the scaling methodology proposed in [33], scaling to a 7nm process would reduce the area to approximately 0.9 mm^2 and the power consumption to 2.1 W, making ABC-FHE highly feasible for client-side applications.

B. Evaluation Setup

To evaluate the performance of ABC-FHE, we employed a two-step approach: a cycle-level simulator was developed to measure latency, and Design Compiler was used to determine area and power characteristics. For this evaluation, we selected parameters used in CKKS, including a polynomial degree of 2^{16} . The bitwidth of the prime numbers was set to 36-bit, following the double scale technique [21], and the number of levels was doubled from the standard 12 to 24. Additionally, we assumed that messages from the client are encrypted to a

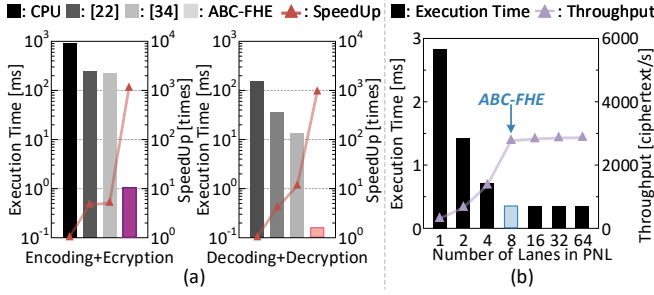


Fig. 5. Performance of ABC-FHE (a) Execution time and speed-up for encoding/encryption and decoding/decryption (b) Effect of number of pipelined NTT lanes on execution time and throughput.

24-level ciphertext, while ciphertexts from the server arrive at a 2-level state to facilitate efficient computation on the server and minimize computational overhead on the client.

C. Results and Discussions

Latency and Throughput. Figure 5 (a) highlights the significant performance improvements of ABC-FHE in terms of execution time and speed-up compared to prior works. Since previous designs do not support bootstrappable parameters, their reported latency was scaled by the proportion of operations for fair comparison. Additionally, to account for platform differences, FPGA and ASIC results were normalized to match ABC-FHE’s 600 MHz frequency. ABC-FHE demonstrated a remarkable latency reduction: for encoding and encryption, it achieved a $1112\times$ reduction compared to a PC-grade Intel Core i7-12700 using Lattigo [34] and a $214\times$ reduction compared to SOTA ASIC and FPGA implementations. For decoding and decryption, ABC-FHE reduced latency by $963\times$ compared to the CPU and $82\times$ compared to state-of-the-art ASIC and FPGA implementations. These results underscore ABC-FHE’s superior efficiency, particularly in handling encoding and encryption tasks with large levels through streaming operations.

Figure 5 (b) shows the impact of varying the number of PNLs on execution time and throughput. Under LPDDR5 specs typically used in client-side applications, the memory bottleneck was observed to cap performance at a maximum of 8 lanes, which ABC-FHE utilizes to maximize acceleration. By leveraging this optimal configuration, ABC-FHE maximized the number of ciphertexts processed per second, achieving peak performance in client-side FHE operations.

Area Optimization in RFE. Figure 6 (a) demonstrates the effectiveness of the proposed optimizations in reducing the hardware area of the RFE. All designs are evaluated using a $P=8$ MDC-based pipelined NTT architecture, comparing the area required to generate one FFT result and four NTT results. For fairness, the comparison assumes two RFEs for cases requiring separate hardware for NTT and FFT computations, with other configurations adjusted accordingly.

The baseline design adopts the commonly used radix-2 pipelined NTT architecture with distinct hardware implementations for NTT and FFT. Optimization techniques include twiddle factor scheduling to reduce the number of multipliers and Montgomery multiplier optimization to minimize the area

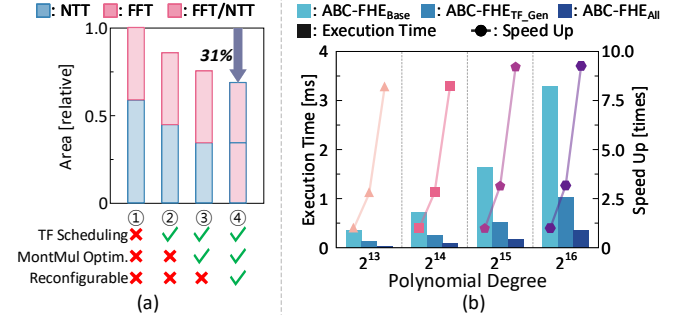


Fig. 6. (a) Area reduction as area optimization applied (b) Execution time improvement as memory optimization applied

of computational units. The final configuration represents a fully reconfigurable RFE capable of supporting both NTT and FFT operations. Combined, these optimizations achieved a 31% reduction in total area, underscoring the efficiency of the proposed approach.

On-chip Memory Optimization. To minimize EMA, ABC-FHE incorporates a unified OTF TF Gen and a PRNG to generate critical data such as twiddle factors, masks, errors, and keys directly on-chip. Figure 6 (b) evaluates the performance gain achieved by this approach across various polynomial degrees. Three configurations were analyzed: ABC-FHE_{Base}, which fetches all necessary data from external memory; ABC-FHE_{TF_Gen}, which generates only twiddle factors on-chip; and ABC-FHE_{All}, which uses both unified OTF TF Gen and PRNG to generate all data on-chip. Compared to external memory reliance, ABC-FHE_{All} achieved a latency reduction of approximately $8.2\text{--}9.3\times$. Given that the combined area of the unified OTF TF Gen and PRNG constitutes only 6% of the total chip area, this design demonstrates significant performance improvements with minimal area overhead.

VI. CONCLUSION

In this work, we proposed ABC-FHE, a resource-efficient client-side accelerator designed to support bootstrappable parameters. Prior works fail to address bootstrappable parameters or rely on non-streaming architectures, leading to memory bottlenecks and latency overhead when throughput increases. To overcome these issues, ABC-FHE employs a streaming architecture for consistent throughput and generates parameters like random values and twiddle factors on-chip, minimizing external memory access. This design achieves significant speedups: $1112\times$ and $214\times$ faster encoding/encryption and $963\times$ and $82\times$ faster decoding/decryption compared to a CPU and the state-of-the-art accelerator, respectively.

ACKNOWLEDGMENT

This work was supported by the MSIT (Ministry of Science and ICT), Korea, through the IITP (Institute for Information & Communications Technology Planning & Evaluation) under the ITRC program (IITP-2025-RS-2020-II201847), the PIM Technology Development project (2022-0-01037), and the Graduate School of AI Semiconductor program (IITP-2025-RS-2023-00256472).

REFERENCES

- [1] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [2] H. Chen, X. Liu, D. Yin, and J. Tang, "A survey on dialogue systems: Recent advances and new frontiers," *Acm Sigkdd Explorations Newsletter*, vol. 19, no. 2, pp. 25–35, 2017.
- [3] V. Kepuska and G. Bohouta, "Next-generation of virtual personal assistants (microsoft cortana, apple siri, amazon alexa and google home)," in *2018 IEEE 8th annual computing and communication workshop and conference (CCWC)*. IEEE, 2018, pp. 99–103.
- [4] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.
- [5] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim *et al.*, "Privacy-preserving machine learning with fully homomorphic encryption for deep neural network," *IEEE Access*, vol. 10, pp. 30 039–30 054, 2022.
- [6] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3–7, 2017, Proceedings, Part I 23*. Springer, 2017, pp. 409–437.
- [7] W. Jung, E. Lee, S. Kim, J. Kim, N. Kim, K. Lee, C. Min, J. H. Cheon, and J. H. Ahn, "Accelerating fully homomorphic encryption through architecture-centric analysis and optimization," *IEEE Access*, vol. 9, pp. 98 772–98 789, 2021.
- [8] J. Kim, G. Lee, S. Kim, G. Sohn, M. Rhu, J. Kim, and J. H. Ahn, "Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1237–1254.
- [9] J. Kim, S. Kim, J. Choi, J. Park, D. Kim, and J. H. Ahn, "Sharp: A short-word hierarchical accelerator for robust and practical fully homomorphic encryption," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.
- [10] A. Putra, Prasetyo, Y. Chen, J. Kim, and J.-Y. Kim, "Strix: An end-to-end streaming architecture with two-level ciphertext batching for fully homomorphic encryption with programmable bootstrapping," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 1319–1331.
- [11] Prasetyo, A. Putra, J.-Y. Kim *et al.*, "Morphling: A throughput-maximized tfft-based accelerator using transform-domain reuse," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2024, pp. 249–262.
- [12] X. Deng, S. Fan, Z. Hu, Z. Tian, Z. Yang, J. Yu, D. Cao, D. Meng, R. Hou, M. Li *et al.*, "Trinity: A general purpose fft accelerator," *arXiv preprint arXiv:2410.13405*, 2024.
- [13] J. Wang, C. Yang, J. Hou, F. Zhang, Y. Meng, Y. Su, and L. Liu, "A compact and efficient hardware accelerator for rms-ckks en/decoding and en/decryption," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2024.
- [14] Z. Azad, G. Yang, R. Agrawal, D. Petrisko, M. Taylor, and A. Joshi, "Race: Risc-v soc for en/decryption acceleration on the edge for homomorphic computation," in *Proceedings of the ACM/IEEE International Symposium on Low Power Electronics and Design*, 2022, pp. 1–6.
- [15] S. Di Matteo, M. L. Gerfo, and S. Saponara, "Vlsi design and fpga implementation of an ntt hardware accelerator for homomorphic seal-embedded library," *IEEE Access*, 2023.
- [16] F. Krieger, F. Hirner, A. C. Mert, and S. S. Roy, "Aloha-he: A low-area hardware accelerator for client-side operations in homomorphic encryption," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–6.
- [17] J.-P. Bossuat, R. Cammarota, J. H. Cheon, I. Chillotti, B. R. Curtis, W. Dai, H. Gong, E. Hales, D. Kim, B. Kumara *et al.*, "Security guidelines for implementing homomorphic encryption," *Cryptology ePrint Archive*, 2024.
- [18] J. Johnston, "Parallel pipeline fast fourier transformer," in *IEEE Proceedings F (Communications, Radar and Signal Processing)*, vol. 130, no. 6. IET, 1983, pp. 564–572.
- [19] T. Ahmed, "A low-power time-interleaved 128-point fft for ieee 802.15.3c standard," in *2013 International Conference on Informatics, Electronics and Vision (ICIEV)*. IEEE, 2013, pp. 1–5.
- [20] M. Garrido, S.-J. Huang, S.-G. Chen, and O. Gustafsson, "The serial commutator fft," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 63, no. 10, pp. 974–978, 2016.
- [21] R. Agrawal, J. H. Ahn, F. Bergamaschi, R. Cammarota, J. H. Cheon, F. DM de Souza, H. Gong, M. Kang, D. Kim, J. Kim *et al.*, "High-precision rms-ckks on fixed but smaller word-size architectures: theory and application," in *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 2023, pp. 23–34.
- [22] M. Garrido, S.-J. Huang, and S.-G. Chen, "Feedforward fft hardware architectures based on rotator allocation," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 2, pp. 581–592, 2017.
- [23] J. K. Jang, H. K. Kim, M. H. Sunwoo, and O. Gustafsson, "Area-efficient scheduling scheme based fft processor for various ofdm systems," in *2018 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*. IEEE, 2018, pp. 338–341.
- [24] M. Garrido and P. Paz, "Optimum mdc fft hardware architectures in terms of delays and multiplexers," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 68, no. 3, pp. 1003–1007, 2020.
- [25] S. S. Roy, F. Vercauteren, N. Mentens, D. D. Chen, and I. Verbauwhede, "Compact ring-lwe cryptoprocessor," in *Cryptographic Hardware and Embedded Systems—CHES 2014: 16th International Workshop, Busan, South Korea, September 23–26, 2014. Proceedings 16*. Springer, 2014, pp. 371–391.
- [26] T. Pöppelmann, T. Oder, and T. Güneysu, "High-performance ideal lattice-based cryptography on 8-bit atxmega microcontrollers," in *International conference on cryptology and information security in Latin America*. Springer, 2015, pp. 346–365.
- [27] P. Barrett, "Implementing the rivest shamir and adleman public key encryption algorithm on a standard digital signal processor," in *Conference on the Theory and Application of Cryptographic Techniques*. Springer, 1986, pp. 311–323.
- [28] P. L. Montgomery, "Modular multiplication without trial division," *Mathematics of computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [29] M. Huang, K. Gaj, S. Kwon, and T. El-Ghazawi, "An optimized hardware architecture for the montgomery multiplication algorithm," in *Public Key Cryptography—PKC 2008: 11th International Workshop on Practice and Theory in Public-Key Cryptography, Barcelona, Spain, March 9–12, 2008. Proceedings 11*. Springer, 2008, pp. 214–228.
- [30] W. Dai, D. D. Chen, R. C. Cheung, and C. K. Koc, "Area-time efficient architecture of fft-based montgomery multiplication," *IEEE Transactions on Computers*, vol. 66, no. 3, pp. 375–388, 2016.
- [31] S. Kim, K. Lee, W. Cho, J. H. Cheon, and R. A. Rutenbar, "Fpga-based accelerators of fully pipelined modular multipliers for homomorphic encryption," in *2019 International Conference on Reconfigurable Computing and FPGAs (ReConFig)*. IEEE, 2019, pp. 1–8.
- [32] T. T. Nguyen, J. Kim, and H. Lee, "Ckks-based homomorphic encryption architecture using parallel ntt multiplier," in *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2023, pp. 1–4.
- [33] S. Sarangi and B. Baas, "Deepscaletool: A tool for the accurate estimation of technology scaling in the deep-submicron era," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2021, pp. 1–5.
- [34] tuneinsight, "Lattigo v6," Online: <https://github.com/tuneinsight/lattigo>, Aug. 2024, ePFL-LDS, Tune Insight SA.